ArgoVault

中文 | English

Tauri 2
Platform
Frontend

離線、多設定檔的密碼管理程式：單一 HTML 前端，以 Tauri 2 打包為 Android App（亦可建置 macOS 版）。
An offline, multi-profile password manager: a single-file HTML frontend packaged with Tauri 2 for Android (macOS build also possible).

中文

簡介

ArgoVault（原名 PyPasswords）是原 Python 命令列程式 `PyPasswords.py` 的 HTML 重寫版。所有資料只存在本機，以主碼加密；前端是單一檔案 `src/index.html`，使用原生 JavaScript，沒有任何外部相依，程式碼中也沒有網路請求（`fetch`、`XMLHttpRequest`、`WebSocket`）。

在瀏覽器直接開啟時，資料存於 `localStorage`；在 Tauri 內執行時，資料存於 App 私有資料目錄，兩種模式依 `window.__TAURI__` 自動切換。

功能

- 多設定檔：建立、解鎖、匯入、刪除；每個設定檔各自擁有一組主碼。
- 紀錄管理：新增、編輯、單筆刪除、多選刪除、依帳號名稱搜尋；欄位為帳戶類型、帳號名稱、密碼、連結帳戶、姓名、備註（多筆），另有選填的網站網址（自動填入比對用）；編輯頁的密碼欄有複製鈕。
- 生成密碼：長度 12 至 64，必須同時含小寫、大寫、數字與符號；可選擇是否顯示強度評估。
- 備份：匯出加密檔`.bin` 與雜湊檔`.sha256`；匯入前顯示檔名、大小、檔頭、偵測到的格式與完整 SHA-256。
- 匯入 CSV：欄位為帳戶類型，帳號名稱，密碼，連結帳戶，姓名，備註，備註欄可用；或；分隔多筆；可另加選填的網址欄。
- 管理資料：轉換加密格式、變更主碼、刪除資料庫。
- 生物辨識解鎖（僅 Android，選用）：每個設定檔可在「更多」啟用，之後解鎖頁出現生物辨識按鈕；主碼永遠可用；查看、新增、編輯、刪除等需再輸入主碼的對話框也可用生物辨識驗證。
- 瀏覽器自動填入（僅 Android，選用）：在系統設定選 ArgoVault 為自動填入服務後，於 Chrome 登入頁點欄位即出現建議，解鎖（主碼或生物辨識）後只列出網址符合的紀錄，點選即填入；每次填入都要解鎖，且不會寫入資料庫。

- 介面語言：繁體中文、English、跟隨系統。
- 設定集中於「設定」對話框，見下方「設定項目」。

加密格式

三種格式皆由檔頭自我描述，沿用原 Python 版的檔案格式：

名稱	檔案配置	金鑰衍生與加密
PWV1（Fernet）	`SALT(16)` + Fernet token	PBKDF2-HMAC-SHA256，600000 次
PWV2	`"PWV2"` + `SALT(16)` + `NONCE(12)` + 密文	PBKDF2-HMAC-SHA256，600000 次，AES-256-GCM
PWV3（預設）	`"PWV3"` + `0x01` + `SALT(16)` + `NONCE(12)` + 密文	Argon2id（19456 KiB、2 次迭代、1 lane），AES-256-GCM

明文為 JSON：`{"records":[...]}`。Argon2id 為自行以 JavaScript 實作，執行於 Web Worker，運算期間畫面不會凍結，但總運算時間不會比主執行緒快。

安全設計

- 解鎖後記憶體只保留清單中繼資料；查看密碼、編輯、新增、刪除、匯入 CSV、變更主碼、轉換格式都要再輸入一次主碼。「查看密碼前需再次輸入主碼」可在設定中關閉，此時解鎖後會保留不可匯出的金鑰直到鎖定，其餘操作仍須輸入主碼。
- 密碼預設遮罩，顯示後自動隱藏；閒置與切到背景後自動鎖定，時間可在設定中調整。
- 寫入時先寫暫存檔並讀回比對，通過檢查才取代原檔，並保留 `.bak`。
- 刪除用 PIN（選用）：刪除資料庫、單筆刪除、多選刪除時須輸入；PIN 以 PBKDF2-SHA256 雜湊儲存。
- 禁止截圖與螢幕錄影（僅 Android，選用）。
- 生物辨識解鎖（僅 Android，選用）：主碼以需要強生物辨識（Class 3）的 Android Keystore 金鑰加密後存放；新增指紋會使其失效，主碼變更時自動重新註冊或停用。便利功能，強度低於主碼。
- 自動填入（僅 Android，選用）：鎖定時不洩漏任何帳號，只提供需驗證的建議；網域須等於紀錄網址的主機或其子網域才列入預設清單（`evil-example.com` 不符合 `example.com`），其他紀錄須手動展開並顯示警告；切到背景或閒置 90 秒即取消。
- 編輯中切到背景不會取消編輯或鎖定，但密碼欄會改回遮罩，逾 5 分鐘才取消編輯。
- 全頁禁止選取文字，密碼只能用複製鈕取得；複製後可選擇是否提醒清除剪貼簿。

設定項目

語言、預設加密格式、解鎖時自動轉換為預設格式、啟用密碼強度檢查、生成密碼預設長度、禁止截圖與螢幕錄影、閒置自動鎖定、切到背景後鎖定、查看密碼顯示時間、查看密碼前是否再次輸入主碼、自動填入的帳號欄位（僅 Android）、複製密碼後提醒清除剪貼簿、刪除用 PIN。設定存於 `settings.json`，不加密，PIN 只存雜湊。

在瀏覽器執行

```
git clone https://github.com/HaveLightII/ArgoVault.git
cd ArgoVault
open src/index.html
```

瀏覽器版資料綁定該瀏覽器與網址來源，更換瀏覽器或清除網站資料即遺失，請定期匯出備份。

建置

需求：Node.js、Rust（含 `aarch64-linux-android` 目標）、JDK 17、Android SDK 與 NDK（目前使用 NDK `27.2.12479018`）。

```
export ANDROID_HOME="$HOME/Android/Sdk"
export NDK_HOME="$ANDROID_HOME/ndk/27.2.12479018"
cd ArgoVault
npm install
npm run tauri android build -- --apk --target aarch64
```

輸出為 `src-tauri/gen/android/app/build/outputs/apk/universal/release/app-universal-release-unsi`
需自行對齊並簽署後才能安裝（例如以 uber-apk-signer）。改了 `src/index.html` 必須重新建置；建置時會自動複製一份給自動填入畫面（`assets/argovault-af.html`，為建置產物，不納入版控）。

macOS 版：

```
npm run tauri build -- --bundles dmg
```

Android 版已在三星 arm64 實機測試；macOS 版可建置，但尚未完整實測。

專案結構

路徑	說明
`src/index.html`	全部前端（介面、加密、儲存層）
`src-tauri/`	Tauri 2 專案（Rust、設定、權限、圖示）

路徑	說明
src-tauri/gen/android/	Android 專案，含 MainActivity.kt（狀態列內縮、禁止截圖、返回鍵）、PwBio.kt（生物辨識橋接）、ArgoAutofillService.kt 與 AutofillAuthActivity.kt（自動填入服務與驗證畫面）、res/layout/autofill_item.xml（建議按鈕版面）
ArgoVault_HANDOFF.md	開發交接文件

已知限制

- JavaScript 無法保證覆寫字串記憶體，只能縮短存留時間。
- 刪除用 PIN 只防誤刪與旁人操作，不是存取控制；設定檔不加密。
- 解除安裝 App 會清空資料；更換簽章前務必先匯出備份。
- 搜尋僅比對帳號名稱；備註尚未支援逐筆編輯。
- 自動填入：只針對 Chrome；不支援先填帳號、下一頁才填密碼的多步驟登入；不使用公共尾碼清單，網址不要填 co.uk 這類；填入的帳號取自「帳號名稱」或「連結帳戶」(設定可選)。

路線圖

- 以 WebAssembly 版 Argon2 取代 JavaScript 實作以縮短運算時間。
- 備註逐筆編輯。
- 記住上次使用的設定檔。

作者

- @HaveLightII

授權

GPL-3.0 license

回到頂端

English

Overview

ArgoVault (formerly PyPasswords) is an HTML rewrite of the original Python command-line tool `PyPasswords.py`. All data stays on the device and is encrypted with a master password. The frontend is a single file, `src/index.html`, written in vanilla JavaScript with no external dependencies and no network requests (`fetch`, `XMLHttpRequest`, `WebSocket`).

Opened directly in a browser, data is kept in `localStorage`; inside Tauri it is stored in the app' s private data directory. The mode is chosen automatically by checking `window.__TAURI__`.

Features

- Multiple profiles: create, unlock, import and delete; each profile has its own master password.
- Record management: add, edit, delete, multi-select delete, and search by account name. Fields are account type, account name, password, linked account, name and notes (multiple), plus an optional website URL used for autofill matching; the edit page has a copy button for the password.
- Password generator: length 12 to 64, always containing lowercase, uppercase, digits and symbols; strength rating can be turned off.
- Backup: export the encrypted `.bin` plus a `.sha256` file; before importing, the file name, size, header bytes, detected format and full SHA-256 are shown.
- CSV import: columns 帳戶類型, 帳號名稱, 密碼, 連結帳戶, 姓名, 備註 (the Chinese headers are required); the notes column may hold several notes separated by ; or ；; an optional 網址 column may be added.
- Data management: convert encryption format, change the master password, delete a database.
- Biometric unlock (Android only, optional): enable it per profile under More; the unlock screen then shows a biometric button. The master password always works; the re-authentication dialogs (viewing, adding, editing, deleting and so on) also accept biometric verification.
- Browser autofill (Android only, optional): after choosing ArgoVault as the autofill service in system settings, tapping a field on a Chrome login page shows a suggestion; after unlocking (master password or biometrics) only records whose URL matches are listed, and one tap fills the fields. Every fill requires unlocking and nothing is written to the database.
- UI language: Traditional Chinese, English, or follow the system.
- Preferences are collected in one Settings dialog (see "Settings" below).

Encryption formats

Each format describes itself in the file header and follows the file formats of the original Python version:

Name	File layout	Key derivation and cipher
PWV1 (Fernet)	`SALT(16)` + Fernet token	PBKDF2-HMAC-SHA256, 600000 iterations
PWV2	`"PWV2"` + `SALT(16)` + `NONCE(12)` + ciphertext	PBKDF2-HMAC-SHA256, 600000 iterations, AES-256-GCM
PWV3 (default)	`"PWV3"` + `0x01` + `SALT(16)` + `NONCE(12)` + ciphertext	Argon2id (19456 KiB, 2 passes, 1 lane), AES-256-GCM

The plaintext is JSON: `{"records":[...]}`. Argon2id is a hand-written JavaScript implementation that runs in a Web Worker: the UI no longer freezes during the computation, but total time is not shorter than on the main thread.

Security design

- After unlocking, only list metadata is kept in memory; viewing a password, editing, adding, deleting, importing CSV, changing the master password and converting the format each ask for the master password again. The check before viewing a password can be turned off in Settings; the app then keeps a non-extractable key in memory until it locks, and every other operation still asks for the master password.

- Passwords are masked by default and hidden again automatically; the app locks after idle time or after being in the background, both configurable.

- Writes go to a temporary file first and are read back and compared before replacing the original, which is kept as `.bak`.

- Optional deletion PIN: required to delete a database, a single record or multiple records; stored as a PBKDF2-SHA256 hash.

- Optional screenshot and screen-recording block (Android only).

- Optional biometric unlock (Android only): the master password is encrypted with an Android Keystore key that requires a strong (Class 3) biometric; enrolling a new fingerprint invalidates it, and changing the master password re-enrolls or disables it. A convenience feature, weaker than the master password.

- Autofill (Android only, optional): while locked it exposes no accounts, only an authentication-required suggestion; the request domain must equal the record URL host or be a subdomain of it (`evil-example.com` does not match `example.com`), other records must be revealed manually with a warning; it is cancelled when the app goes to the background or after 90 seconds idle.

- Going to the background while editing neither cancels the edit nor locks the vault, but the password field is masked again and the edit is cancelled after 5 minutes.

- Text selection is disabled across the page, so passwords can only be taken with the copy button; a clipboard-cleanup reminder after copying is optional.

Settings

Language, default encryption format, auto-convert to the default format on unlock, password strength check, default generated length, block screenshots and screen recording, idle auto-lock, lock after background, password reveal time, re-entering the master password before revealing a password, the field used as the autofill username (Android only), clipboard reminder after copying, and the deletion PIN. Settings are stored in `settings.json`, unencrypted; only a hash of the PIN is stored.

Run in a browser

```
git clone https://github.com/HaveLightII/ArgoVault.git
cd ArgoVault
open src/index.html
```

Browser data is tied to that browser and origin; switching browsers or clearing site data loses it, so export backups regularly.

Build

Requirements: Node.js, Rust, JDK 17, Android SDK and NDK (currently NDK `27.2.12479018`); the Rust target `aarch64-linux-android` is also needed.

```
export ANDROID_HOME="$HOME/Android/Sdk"
export NDK_HOME="$ANDROID_HOME/ndk/27.2.12479018"
cd ArgoVault
npm install
npm run tauri android build -- --apk --target aarch64
```

The output is `src-tauri/gen/android/app/build/outputs/apk/universal/release/app-universal-releas` align and sign it yourself before installing (for example with uber-apk-signer). Changing `src/index.html` requires a rebuild; the build also copies it for the autofill screen (`assets/argovault-af.html`, a build artifact that is not tracked in git).

macOS:

```
npm run tauri build -- --bundles dmg
```

The Android build has been tested on a Samsung arm64 device; the macOS build is buildable but not yet fully tested.

Project layout

Path	Description
`src/index.html`	The entire frontend (UI, cryptography, storage layer)

Path	Description
`src-tauri/`	Tauri 2 project (Rust, configuration, permissions, icons)
`src-tauri/gen/android/`	Android project, including `MainActivity.kt` (status bar insets, screenshot block, back key), `PwBio.kt` (biometric bridge), `ArgoAutofillService.kt` and `AutofillAuthActivity.kt` (autofill service and authentication screen), and `res/layout/autofill_item.xml` (suggestion layout)
`ArgoVault_HANDOFF.md`	Development handoff document (Traditional Chinese)

Known limitations

- JavaScript cannot guarantee that string memory is overwritten; it can only shorten how long secrets stay around.
- The deletion PIN only guards against accidental or casual deletion; it is not access control, and the settings file is unencrypted.
- Uninstalling the app wipes its data; always export a backup before changing the signing key.
- Search matches the account name only; notes cannot yet be edited one by one.
- Autofill: Chrome only; multi-step logins (username on one page, password on the next) are not supported; no public-suffix list is used, so do not enter URLs such as `co.uk`; the username filled comes from the account name or the linked account (selectable in Settings).

Roadmap

- Replace the JavaScript Argon2id with a WebAssembly build to shorten computation time.
- Per-note editing.
- Remember the last used profile.

Author

- @HaveLightII

License

GPL-3.0 license

Back to top